

集合与字典

• 集合 (set): 若干不重复元素的一个群集 (元素间无关系, 无顺序)

(一). 用位向量实现集合的 ADT:

将全集 U 存入一个二进制数组 $A[]$ 中, 若 $A[i]=1$, 表示在集合 A 中, 否则不在 A 中.

• 集合运算实现: 若 $A, B \subseteq U$, 分别存在 $A[]$ 和 $B[]$ 中, 则:

$A+B$: $A[i] \mid B[i]$ 按位运算 (i 从 0 到 $\maxSize$)

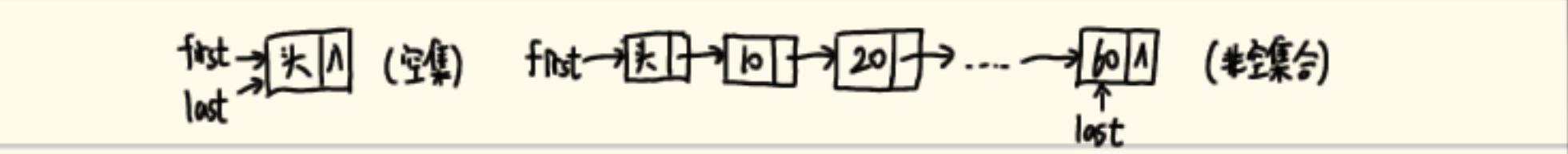
AB : $A[i] \& B[i]$ 按位运算 (i 从 0 到 $\maxSize$)

$A-B$: $A[i] \& !B[i]$ 按位运算 (i 从 0 到 $\maxSize$)

各操作 (增, 删, 改, 查) 均为 $O(1)$ 时间, 且占用空间极小

(二). 用有序链表实现:

• 不用知道全集, 只存需要的子集部分, 且事先排序好 提高搜索效率



• 集合运算实现: 用 $*pa = A \rightarrow first$, $*pb = B \rightarrow first$, 然后比 $*pa$ 和 $*pb$

元素的大小, 再按运算需要存入新链表中 (尤其注意 "+" 时, 两个集合共有的只取一个)

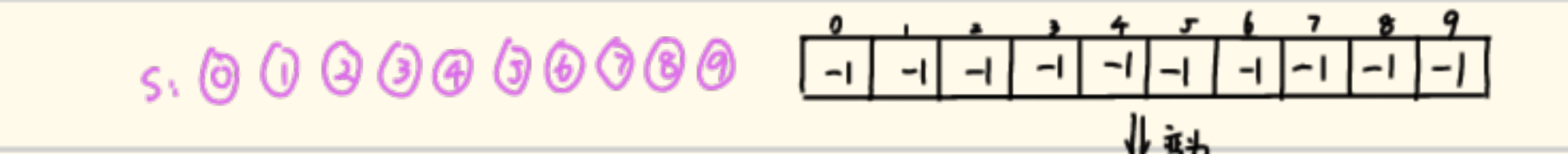
• 并查集 (UFS): 支持合并、查找操作的集合 (采用树的双亲表示作为存储方式)

实现: 每个集合以一棵树表示, 树的每一个结点代表集合中的一个单元, 所有集合的全集

构成一个森林。同时用一个数组 A , $A[i]$ 代表包含集合元素 i 的树结点, 若其为负,

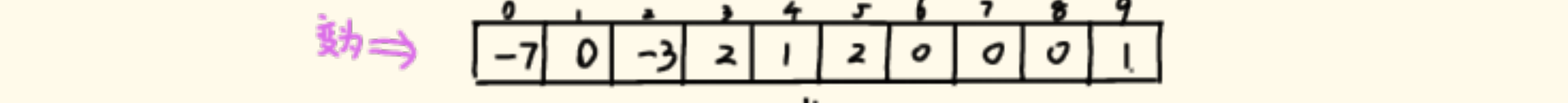
表示其及其各个元素成一个子集, 大小为 $|A[i]|$; 否则表示该元素属于 $A[A[i]]$ 的这一子集

eg: $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, 初始化时每个元素自成一个子集:



操作后 $S' = \{S_1\{0, 6, 7, 8\}, S_2\{1, 2, 3, 5\}, S_3\{2, 3, 5\}\}$

要实现 $S_1 \cup S_2$, 只要将其中一个根节点当另一个的子女即可



* 并查集代码实现 (主要针对 $parent[]$ 数组, 即可完成集合操作)

1. 查: 元素 x 的树的根

Find (int x) { while (parent[x] > 0) x = parent[x]; return x; } // 只有树根的 parent 为负值

2. 并: 将 $root1$ 和 $root2$ 这两个子集合的根合并为其一

Set_Union (int root1, int root2) { parent[root1] += parent[root2]; parent[root2] = root1; }

* 若有 n 个元素, 每个元素查找为 $O(1)$, 故查找所有元素最坏为 $O(n^2)$, 需优化

优化方法: 小树并大树 (加权规则)、压缩树高、压缩路径

• 应用: 等价类: 集合上的一个自反、传递、对称的关系

* 用并查集实现: 先初始化一个并查集 (所有元素独立成一类); 然后每接受一对等价关系, 就

Find 对应两个等价结点, 将其 Union 一下。直至输入流结束, 即得若干个集合, 就是等价类

• 字典 (dictionary): $\langle \text{名字} - \text{属性} \rangle$ 对的集合

* 需要实现操作: 确定一个名字是否在字典中; 输出/修改一个名字的属性; 插入; 删除

(一). 字典的线性表实现:

按 name 从小到大的顺序存储 (name 怎么比大小看具体数据类型)

• 数组: 增/删/改/查 均从 $i=0$ 往后遍历, 若 $Dict[i].name$ 符合要求则返回; 若 $Dict[i].name >$ 要求则 break;

后面不用遍历了。平均搜索长度 $ASL_{succ} = \sum_{i=1}^n P_i C_i = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$, $ASL_{unsucc} = \frac{1}{n+1} (\sum_{i=1}^n i + n)$

* 将 $O(n)$ 优化成 $O(\log_2 n)$: 折半查找, 接收参数 low, high 表示查找区域, 先取 $mid = \frac{low+high}{2}$, 然后若

$Dict[mid] <$ 要求, 则递归查找后一半 ($mid+1, high$); 若 $Dict[mid] >$ 要求, 则递归查找前一半; 若相等则返回即可。

• 链表: 增/删/改/查 均从头节点 往后遍历, 复杂度均为 $O(n)$

• 散列表: 将关键码映射到表中某个位置上, 然后根据关键码用同样的方式直接访问 $Address = hash(key)$

* 散列函数 hash: 关键码集合 $\rightarrow$ 散列表地址集合

* 冲突: 两个不同关键码经 $hash()$ 处理得到相同的地址, 即 "同义词", 需特殊处理

(一). hash 函数的选择:

1. 直接线性映射: $kx+b$ 等函数, 缺点是实际上并未压缩, 地址多。

2. 数字分析法: 事先分析 key 每一位的数字均匀度, 然后取均匀的若干位做地址。

3. 平方取中法: 先把 key 转换为内码, 然后平方后, 再取中间若干位作为地址。

4. 除留余数法: 设最大允许地址为 m , 取一个小于 m 的最大质数 p , $hash(key) = key \% p$ (常用)

5. 折叠法: key 较大时, 将其分为若干段 (每段长度为允许地址位数), 然后逐位相加得出地址

(二) 冲突的处理:

1. 线性探查法: 插入表项时先搜索 $H_0 = hash(key)$, 若已有元素则说明冲突了, 不插入, 线性向后

移直至找到空格, 然后插入即可。

eg: 已知散列表 $A[0 \dots 11]$, $hash(k) = k \% 11$, 关键字集合为 $\{10, 20, 15, 17, 21, 6, 8, 25, 40, 35, 27\}$

解: $hash(key) = \{10, 9, 4, 6, 10, 6, 8, 3, 7, 2, 5\}$



平均搜索长度 $ASL_{succ} = \frac{1}{11} \times (6+1+1+1+1+1+2+1+1+1+2) = \frac{18}{11}$

$ASL_{unsucc} = \frac{1}{11} \times (2+1+2+1+1+\dots+4) = \frac{25}{11}$

ASL_{succ} : 每一个表项向后移动次数 + 1 的平均值

ASL_{unsucc} : 能通过 Hash 函数得到的地址上向后移动到空格的次数 + 1 的平均值

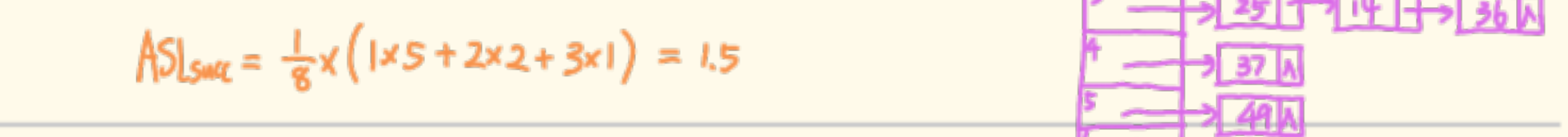
* 缺点: 删除某元素会影响其他元素的搜索, 故不适用于散列表经常变的情况

2. 二次探查法、双散列法 (*不做要求)

3. 开散列法 (也称链地址法): 每个位置 ($0 \sim m-1$) 放一个桶, 存放所有同义词。

eg: 已知散列表 $HT(12)$, $Hash(x) = x \% 11$, 一组元素关键码为 $37, 25, 14, 36, 49, 68, 57, 11$ 。

则散列表结构为:



$ASL_{succ} = \frac{1}{8} \times (1 \times 5 + 2 \times 2 + 3 \times 1) = 1.5$

$ASL_{unsucc} = \frac{1}{11} \times (2+1+3+4+2+2+1 \times 5) = \frac{19}{11}$